

Data Analysis Project Report: Customer Sales & Churn Analysis

Project Description

This project represents an advanced relational data analysis pipeline built using Python. By leveraging the pandas library, disconnected sales and customer retention databases are cleaned, transformed, and merged. The analysis focuses on extracting deep insights into purchasing behavior, localized cross-selling patterns, and customer churn vulnerabilities, culminating in a multi-chart executive dashboard generated via matplotlib.

Objectives

- Build an advanced data manipulation pipeline: Clean -> Extract -> Merge -> Pivot -> Visualize.
- Solve primary key mismatches between datasets using string manipulation.
- Perform time-series grouping by extracting months and years from raw datetime strings.
- Utilize matplotlib.pyplot.subplots to generate a unified 4-chart executive dashboard.
- Extract and document meaningful business insights regarding customer lifetime value and retention.

Technical Requirements Implemented

- **String Operations & Cleaning:** Used `.str.replace()` to strip alphabetical characters from mismatched IDs, allowing for a clean relational database join.
- **Relational Data Merging:** Implemented `pd.merge(how='inner')` to combine sales transactions with customer profiles based on a unified primary key.
- **Advanced Aggregation:** Utilized `pd.pivot_table()` to create multidimensional summaries (Region vs. Product) and applied multi-conditional filtering (`&`) to isolate VIP customer segments.
- **Subplot Visualization:** Integrated `plt.subplots(2, 2)` to render a 2x2 grid containing line charts, horizontal bar charts, vertical bar charts, and scatter plots, saving the unified dashboard as a high-resolution image.

Data Manipulation & Visualization Logic

The program utilizes the following advanced pandas logic to process and visualize the relational data:

Operation Performed	Python Logic Used	Business Insight / Purpose
Data Merging	<code>pd.merge(sales, churn, on='CustNum', how='inner')</code>	Combines transactional data with demographic data to see who is buying what.
Pivot Table Summary	<code>pd.pivot_table(df, index='Region', columns='Product', aggfunc='sum')</code>	Identifies cross-selling patterns, revealing which specific products dominate in which specific regions.
Multi-Condition Filtering	<code>df[(df['Total_Sales'] > 100k) & (df['Tenure'] >= 12)]</code>	Isolates the "VIP Segment" to prove that high-value, long-term customers have a 0% churn rate.
Dashboard Grid	<code>fig, axs = plt.subplots(2, 2)</code>	Consolidates monthly trends, product performance, top customers, and churn risk into a single executive view.



Setup & Installation

1. Install Python on your system
2. Install required libraries by running: `pip install pandas matplotlib jupyter`
3. Download or clone this repository
4. Ensure `sales_data.csv` and `customer_data.csv` are placed in the root directory
5. Run the notebook using Jupyter, or execute the script using:
`python customer_analysis.py`



Sample Input & Output

The pipeline processes two separate datasets. Here is a sample of the raw data being joined:

Sales Data:

Date: 2024-01-01 | Product: Phone | Customer_ID: CUST001 | Total_Sales: 261100

Customer Data:

CustomerID: C00001 | Tenure: 6 | MonthlyCharges: 64 | Churn: 0

Result for Sample Data:

=====

 Datasets merged successfully. Total records: 100



QUICK METRICS:

Total Revenue: ₹12,365,048.00

Average Order: ₹123,650.48

Top Customer: CUST016 (₹373,932.00)

🎨 Generating Sales Dashboard...

✅ Dashboard saved successfully in visualizations/executive_dashboard.png



📁 Project Files

- customer_analysis.ipynb – Main Python data pipeline notebook
- analysis_report.docx – Final business analysis and executive summary
- sales_data.csv – Raw transactional dataset
- customer_data.csv – Raw customer retention dataset
- visualizations/executive_dashboard.png – Output 4-Chart Dashboard
- requirements.txt – Dependencies (pandas, matplotlib, jupyter)

📖 What I Learned

- Perform relational database operations in Python, specifically merging multiple datasets together using primary and foreign keys.
- Clean messy data using string manipulation to ensure IDs match across different systems.
- Create pivot tables programmatically to quickly summarize multi-dimensional business data.

- Build complex, multi-chart visual dashboards using matplotlib subplots, transitioning from single-chart scripts to professional reporting tools.